



PyTorch Cheat Sheet

Tensors → Autograd → Models → Training · a complete quick reference

Amer Hussein

[linkedin.com/in/aamero](https://www.linkedin.com/in/aamero)

FOUNDATIONS BUILD TRAIN

keyword string number function

SYNTAX

builtin comment

01 Setup & Device

FOUNDATIONS

```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.__version__      # '2.x'
torch.manual_seed(0)   # reproducibility

# pick the best available device
device = (
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
```

02 Creating Tensors

```
torch.tensor([[1, 2], [3, 4]]) # from data
torch.zeros(2, 3)              # 2x3 of 0.
torch.ones(2, 3); torch.eye(3) # ones / identity
torch.full((2, 2), 7)         # constant fill
torch.arange(0, 10, 2)         # 0 2 4 6 8
torch.linspace(0, 1, 5)       # 5 pts, ends incl
torch.rand(2, 3)              # uniform [0,1)
torch.randn(2, 3)             # normal N(0,1)
torch.randint(0, 9, (2, 2))    # random ints
torch.from_numpy(arr)         # share memory
torch.zeros_like(x)           # match shape+dtype
```

03 Attributes & Dtypes

```
x.shape      # torch.Size([2, 3])
x.dtype      # torch.float32
x.device     # cpu / cuda:0
x.ndim, x.numel() # rank, n elements
x.requires_grad # is it tracked?

x.float(); x.long(); x.bool() # cast dtype
x.to(torch.float16)          # half precision
x.to(device, dtype=torch.float32)
```

Default float is `float32`; default int is `int64`.

04 Indexing & Slicing

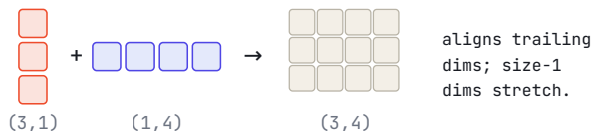
```
x[0]          # first row
x[:, 1]       # column 1, all rows
x[1:, :2]     # slice rows & cols
x[x > 0]      # boolean mask → 1-D
x[..., -1]    # last on final dim
x.gather(1, idx) # gather along dim=1
torch.where(c, a, b) # elementwise select
x.masked_fill(mask, 0)
```

05 Reshaping & Layout

```
x.view(6)      # needs contiguous mem
x.reshape(2, 3) # safe general reshape
x.flatten(start_dim=1)
x.unsqueeze(0)  # add axis → (1, ...)
x.squeeze()     # drop size-1 axes
x.permute(2, 0, 1) # reorder all axes
x.transpose(0, 1) # swap two axes
torch.cat([a, b], dim=0) # join, same ndim
torch.stack([a, b])      # join on NEW axis
```

06 Math & Broadcasting

```
a + b; a * b; a ** 2      # elementwise
a @ b; torch.matmul(a, b) # matrix product
a.sum(dim=0); a.mean(dim=1) # reductions
vals, idx = a.max(dim=1)   # values + argmax
a.argmax(dim=1)
torch.exp(a); torch.sqrt(a); a.clamp(0, 1)
```



07 NumPy Bridge & GPU

```
a = t.numpy()           # tensor → ndarray (shared)
t = torch.from_numpy(a) # ndarray → tensor (shared)

t = t.to(device)        # move to gpu/cpu
t = t.cuda(); t = t.cpu()
t.detach().cpu().numpy() # safe export for logging
```

CPU tensor & array share memory — edits propagate both ways.

08 Autograd

BUILD

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x          # builds the graph
y.backward()                # compute dy/dx
x.grad                     # → tensor(7.)

with torch.no_grad():       # stop tracking
    w -= lr * w.grad        # manual update
z = x.detach()              # cut from graph
x.grad.zero_()              # clear before reuse
```



09 Define a Model

```
class Net(nn.Module):
    def __init__(self, in_d, hid, out_d):
        super().__init__()
        self.fc1 = nn.Linear(in_d, hid)
        self.fc2 = nn.Linear(hid, out_d)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)

model = Net(784, 256, 10).to(device)
```

```
# quick stack alternative
model = nn.Sequential(
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, 10),
)
```

10 Common Layers

```
nn.Linear(in_f, out_f)
nn.Conv2d(in_ch, out_ch, k, stride, padding)
nn.MaxPool2d(2); nn.AvgPool2d(2)
nn.BatchNorm2d(ch); nn.LayerNorm(dim)
nn.Dropout(p=0.5)
nn.Embedding(vocab, dim)
nn.LSTM(in_f, hid, layers, batch_first=True)
nn.MultiheadAttention(embed, heads)
nn.Flatten()
```

11 Activations & Losses

```
# activations (module / functional)
nn.ReLU() → F.relu      nn.GELU() → F.gelu
nn.Sigmoid()            nn.Tanh()
nn.Softmax(dim=-1)      F.log_softmax

# losses
nn.MSELoss()             # regression
nn.CrossEntropyLoss()    # logits + class idx
nn.BCEWithLogitsLoss()   # binary, stable
nn.NLLLoss()             # pair w/ log_softmax
```

CrossEntropyLoss wants raw logits — do not apply softmax first.

12 Optimizers & Schedulers

```
opt = torch.optim.Adam(model.parameters(),
                        lr=1e-3, weight_decay=1e-4)
torch.optim.SGD(p, lr=0.1, momentum=0.9)
torch.optim.AdamW(p, lr=3e-4)

opt.zero_grad(); loss.backward(); opt.step()

sched = torch.optim.lr_scheduler.\
    CosineAnnealingLR(opt, T_max=epochs)
sched.step() # once per epoch
```

13 Dataset & DataLoader

```
from torch.utils.data import Dataset, DataLoader

class DS(Dataset):
    def __init__(self, X, y): self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, i):
        return self.X[i], self.y[i]

loader = DataLoader(DS(X, y), batch_size=64,
                    shuffle=True, num_workers=4)
for xb, yb in loader:
    ...
```



```

model.train() # enable dropout / BN
for epoch in range(epochs):
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)

        pred = model(xb) # 1. forward pass
        loss = criterion(pred, yb) # 2. compute loss

        optimizer.zero_grad() # 3. clear old grads
        loss.backward() # 4. backprop
        optimizer.step() # 5. update weights

    scheduler.step() # per-epoch LR update
  
```

```

# ---- validation / evaluation ----
model.eval() # freeze BN / dropout
correct = 0
with torch.no_grad(): # no graph → faster
    for xb, yb in val_loader:
        logits = model(xb.to(device))
        preds = logits.argmax(dim=1)
        correct += (preds == yb.to(device)).sum().item()

acc = correct / len(val_loader.dataset)
  
```

```

# recommended: save the state_dict only
torch.save(model.state_dict(), "model.pt")
model.load_state_dict(torch.load("model.pt"))
model.eval()

# full checkpoint (resume training)
torch.save({"epoch": e,
            "model": model.state_dict(),
            "opt": opt.state_dict()}, "ckpt.pt")

# inference
with torch.no_grad():
    y = model(x.to(device)).argmax(dim=1)
  
```

```

# count trainable parameters
sum(p.numel() for p in model.parameters()
    if p.requires_grad)

# freeze a submodule
for p in model.fc1.parameters():
    p.requires_grad = False

# move a batch dict to device
batch = {k: v.to(device) for k, v in batch.items()}

# gradient accumulation over N steps
(loss / N).backward()
if step % N == 0:
    opt.step(); opt.zero_grad()
  
```

```

from torch.amp import autocast, GradScaler
scaler = GradScaler() # fp16 loss scaling

for xb, yb in loader:
    opt.zero_grad()
    with autocast(device_type="cuda"): # auto fp16/bf16
        loss = criterion(model(xb), yb)
    scaler.scale(loss).backward()
    scaler.step(opt); scaler.update()

model = torch.compile(model) # 2.x JIT speedup
  
```

- **Zero your grads** each step — `opt.zero_grad()`; grads accumulate by default.
- Use `loss.item()` to log a Python float (it detaches from the graph).
- `model.train()` vs `model.eval()` flips BatchNorm & Dropout behaviour.
- Keep **model and data on the same device**, or you get a device-mismatch error.
- Call `.detach()` before `.numpy()` on tracked tensors.
- Wrap eval in `torch.no_grad()` → less memory, faster.
- Clip exploding grads: `nn.utils.clip_grad_norm_(params, 1.0)`.